



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: HTTP Analysis Using Wireshark: Text

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 10, 2026

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Assignment Title: HTTP Analysis Using Wireshark – Text Traffic

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris, AMCPN

Date: September 10, 2026

EXECUTIVE SUMMARY

This laboratory report documents the practical execution of network traffic acquisition, protocol encapsulation inspection, TCP state reconstruction, and evidence preservation. Operating within a Kali Linux forensic workstation environment ,plaintext Hypertext Transfer Protocol (HTTP) sessions were generated and captured across the local loopback interface (lo) using curl and tshark.

The primary objective of this laboratory exercise is to analyze the forensic value of protocol metadata at the Ethernet, IP, TCP, and HTTP layers. By capturing raw packet data, computing cryptographic hashes, and reconstructing the TCP three-way handshake and connection closure, a repeatable chain of custody was established.

LAB ENVIRONMENT & TOOLING INVENTORY

TOOL / DEPENDENCY	PRIMARY FUNCTION IN FORENSIC WORKFLOW
LINUX (KALI WORKSTATION)	Primary forensic investigation environment (garba@kali).
APACHE2	Local HTTP web server used to host the target HTML evidence page.
CURL	Command-line HTTP client used to generate controlled web requests.
WIRESHARK / TSHARK	Packet capture and network protocol analysis engines.
SHA256SUM	Cryptographic hashing utility used for capture preservation and integrity validation.

SECTION A: THEORETICAL OVERVIEW & CORE CONCEPTS

1. HTTP Protocol Encapsulation & Metadata Value

Network forensics relies on the OSI model encapsulation process, where application-layer data (HTTP) is progressively wrapped in Transport (TCP), Network (IP), and Data Link (Ethernet) headers.

- **HTTP Layer:** Reveals application intent, including requested URIs, Host headers, User-Agent strings, server banners, and response status codes.
- **TCP Layer:** Provides session state tracking, source/destination port identification, and relative/absolute sequence numbers that prove data sequence and integrity.

- **IP & Ethernet Layers:** Establish logical source/destination addresses and hardware routing metadata.

2. Loopback Traversal vs. Physical Network Capture

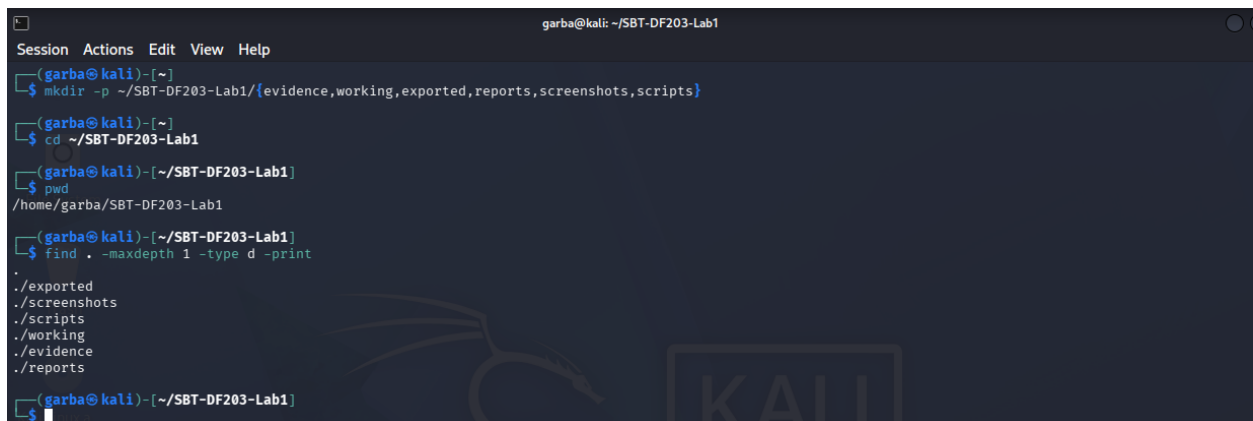
When network traffic travels across the local loopback interface (127.0.0.1 / lo), operating system kernel optimizations bypass physical Network Interface Cards (NICs) and Ethernet driver stacks. Consequently, frames captured on loopback exhibit synthetic/null MAC addresses (00:00:00:00:00:00). True physical hardware layer (L2) evidence recovery requires multi-host virtual networking (e.g., Host-Only adapter configurations).

SECTION B: PRACTICAL LAB EXECUTION & EVIDENTIARY ANALYSIS

TASK 1: ENVIRONMENT PREPARATION & WORKSPACE INITIALIZATION

Part A: Directory Hierarchy & Service Verification

A forensic directory structure was created under ~/SBT-DF203-Lab1 to maintain strict segregation between original evidence, analysis working copies, exported transcripts, and screenshots:

A terminal window titled 'garba@kali: ~/SBT-DF203-Lab1' showing a series of commands and their outputs. The commands are: 'mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}', 'cd ~/SBT-DF203-Lab1', 'pwd' (output: /home/garba/SBT-DF203-Lab1), and 'find . -maxdepth 1 -type d -print' (output: ./exported, ./screenshots, ./scripts, ./working, ./evidence, ./reports).

```
garba@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
(garba@kali)~$ mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
(garba@kali)~$ cd ~/SBT-DF203-Lab1
(garba@kali)~/SBT-DF203-Lab1$ pwd
/home/garba/SBT-DF203-Lab1
(garba@kali)~/SBT-DF203-Lab1$ find . -maxdepth 1 -type d -print
.
./exported
./screenshots
./scripts
./working
./evidence
./reports
(garba@kali)~/SBT-DF203-Lab1$
```

Terminal session verifying the creation of the required directory structure (evidence, working, reports, screenshots).

The Apache web service was started and verified listening on TCP port 80:

Terminal session verifying Apache2 service status (active/running) and socket statistics confirming TCP port 80 listening state.

The target test page containing student identification was authored directly into /var/www/html/basic.html:

Executing printf and tee to populate basic.html with student details.

```

[garba@kali]~$ curl --no-keepalive -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 49572
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 14:40:57 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 14:32:44 GMT
< ETag: "8d-65b21d749b02f"
< Accept-Ranges: bytes
< Content-Length: 141
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-OF203 HTTP Evidence</h1><p>Name: GARBA AHMED</p><p>Reg No: Student ID: 2025/FWSD/11463</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

```

Execution of curl -v demonstrating full HTTP request/response exchange and display of student details.

TASK 2: TRAFFIC ACQUISITION & CHAIN-OF-CUSTODY

Part A: Packet Capture Execution

Packet acquisition was initiated on the loopback interface (lo) filtering specifically for TCP port 80 traffic:

Bash

Terminal 1 (Capture Engine)

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
```

Terminal 2 (Traffic Generation)

```
curl --no-keepalive -v http://127.0.0.1/basic.html
```

```
(garba@kali)~[~/SBT-DF203-Lab1]
$ sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
Capturing on 'Loopback: lo'
10
```

Active tshark capture session running on loopback interface lo capturing HTTP traffic.

```
(garba@kali)~[~/SBT-DF203-Lab1]
$ sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
Capturing on 'Loopback: lo'
10
```

Stopping tshark packet capture engine using Ctrl+C.

Part B: Forensic Preservation & Cryptographic Hashing

To prevent contamination of original evidence, permissions were adjusted, a working copy was duplicated, and SHA-256 integrity hashes were calculated:

```
(garba@kali)~[~/SBT-DF203-Lab1]
$ sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
Capturing on 'Loopback: lo'
10 ^C
```

Permission error encountered when copying root-owned capture file before fixing permissions.

Verification of matching SHA-256 cryptographic hashes for basic.pcapng and basic_working.pcapng.

- **Original Capture SHA-256:**
c464a47532ee8472800b87f9d75e9c181493788bfc94c2cf73a1229ca886d025
- **Working Copy SHA-256:**
c464a47532ee8472800b87f9d75e9c181493788bfc94c2cf73a1229ca886d025
- **Integrity Status:** Identical match confirmed.

TASK 3: TCP THREE-WAY HANDSHAKE RECONSTRUCTION

Using tshark, TCP stream 0 was isolated to evaluate session initiation:

Bash

```
tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
```

```
(garba@kali)-[~/SBT-DF203-Lab1]
$ cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng

(garba@kali)-[~/SBT-DF203-Lab1]
$ sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
c464a47532ee8472800b87f9d75e9c181493788bfc94c2cf73a1229ca886d025  evidence/basic.pcapng
c464a47532ee8472800b87f9d75e9c181493788bfc94c2cf73a1229ca886d025  working/basic_working.pcapng

(garba@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian106StatusE
1      2026-09-10T15:40:57.438651518+0100      127.0.0.1      49572      127.0.0.1      80      0x0002      0      0
2      2026-09-10T15:40:57.438770337+0100      127.0.0.1      80      127.0.0.1      49572      0x0012      0      1

(garba@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian106StatusE
4      2026-09-10T15:40:57.439573266+0100      127.0.0.1      49572      127.0.0.1      80      GET      /basic.html      127.0.0.1      curl/8.21.0

(garba@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian106StatusE
6      2026-09-10T15:40:57.447019313+0100      200      OK      Apache/2.4.68 (Debian)      text/html      141
```

tshark output displaying TCP three-way handshake packet sequence with sequence/acknowledgement progression.

TCP Handshake Forensic Breakdown

<i>Packet</i>	<i>Flags</i>	<i>Client Seq</i>	<i>Server Seq/ACK</i>	<i>Timestamp</i>	<i>Forensic Interpretation</i>
<i>Frame 1</i>	SYN (0x0002)	0	0	15:40:57.438651	Client (127.0.0.1:49572) requests session initialization to Server (127.0.0.1:80).
<i>Frame 2</i>	SYN, ACK (0x0012)	0	0 / 1	15:40:57.438770	Server acknowledges client SYN and provides initial sequence number.
<i>Frame 3</i>	ACK (0x0010)	1	1	15:40:57.438780	Client acknowledges server sequence; TCP session established.

TASK 4: HTTP REQUEST & RESPONSE RECONSTRUCTION

Part A: HTTP GET Request Analysis

Filter applied: http.request.

Bash

```
tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
```

- **Frame:** 4
- **Method:** GET
- **Requested URI:** /basic.html
- **Host Header:** 127.0.0.1
- **User-Agent:** curl/8.21.0

Part B: HTTP Server Response Analysis

Filter applied: http.response.

Bash

```
tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
```

- **Frame:** 6
- **Status Code:** 200 OK
- **Server Banner:** Apache/2.4.68 (Debian)
- **Content-Type:** text/html

- **Content-Length:** 141 bytes

TASK 5: CONNECTION CLOSURE & EVIDENCE FINDINGS

Part A: TCP Connection Termination

Filter applied: tcp.flags.fin==1 || tcp.flags.reset==1.

Bash

```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
```

```
(garba@kali) - (~ / SBT-DF203-Lab1)
$ tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv

tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
8      2026-09-10T15:40:57.447727210+0100      127.0.0.1      49572      127.0.0.1      80      0x0011      84      393
9      2026-09-10T15:40:57.448816358+0100      127.0.0.1      80      127.0.0.1      49572      0x0011      393      85
```

Terminal output isolating Frames 8 and 9 showing TCP connection closure via FIN, ACK flags.

Connection teardown was achieved cleanly via two-way FIN-ACK exchange:

1. **Frame 8:** Client sends FIN, ACK (0x0011, Seq 84, Ack 393).
2. **Frame 9:** Server responds with FIN, ACK (0x0011, Seq 393, Ack 85), successfully closing the socket.

REQUIRED FORENSIC FINDINGS SUMMARY

FINDING FIELD	VALUE / OBSERVATION
CAPTURE START AND END TIME	2026-09-10T15:40:57.438651518 to 2026-09-10T15:40:57.448816358+0100
CLIENT IP AND PORT	127.0.0.1 : 49572
SERVER IP AND PORT	127.0.0.1 : 80

CLIENT INITIAL SEQUENCE NUMBER (ISN)	0 (Relative)
SERVER INITIAL SEQUENCE NUMBER (ISN)	0 (Relative)
HTTP REQUEST TIMESTAMP	2026-09-10T15:40:57.439573266+0100
REQUESTED URI	/basic.html
HTTP RESPONSE CODE	200 OK
CONTENT TYPE AND LENGTH	text/html, 141 bytes
CONNECTION TERMINATION METHOD	Clean FIN, ACK sequence (0x0011 flags on Frames 8 and 9)
MAC ADDRESS OBSERVATION	Standard Ethernet MAC addresses are omitted (00:00:00:00:00:00) because loopback (lo) bypasses physical network interface drivers.

FORENSIC LIMITATIONS & LESSONS LEARNED

1. **Unencrypted Plaintext Visibility:** Plaintext HTTP offers complete visibility into packet payloads. Reconstructing TCP streams exposes critical sensitive details including exact URI paths, application server headers, and body content without cryptographic barriers.
2. **Loopback Abstraction Limitations:** Capturing evidence on local loopback (lo) fails to collect Data Link layer (L2) MAC hardware addresses due to kernel-level protocol stack shortcuts.
3. **Immutability via Hash Controls:** Computing SHA-256 hashes immediately post-acquisition ensures working copies remain bit-for-bit identical to original evidence files, preserving chain of custody for legal presentation.